You are a senior AI software architect and full-stack engineer.

I want to build a COMPLETE on-premise RAG (Retrieval-Augmented Generation) web application using Python, FastAPI, Ollama, HTML, CSS, Bootstrap, and JavaScript.

The application must run FULLY OFFLINE except optional URL ingestion.

Assume:

- Ollama is already installed.
- Ollama is already running at the default API endpoint:
    - http://localhost:11434
- The operating system is Windows.
- Anaconda is already installed.
- Development will be done in VS Code.
- Python version is 3.11+
- The application must be lightweight and suitable for:
    - 32 GB RAM
    - Intel CPU
    - Small NVIDIA GPU (optional)
- The application should support local document-based question answering.

I am an engineering student. Therefore:

- Explain every important step.
- Generate code gradually.
- Never skip files.
- Always mention project structure.
- Explain WHY each module exists.
- Explain architecture before coding.
- Keep the code production-oriented but beginner-friendly.

I want a modern ChatGPT-like UI.

The system should support: 1. Uploading documents 2. Indexing documents 3. RAG retrieval 4. Querying local LLM via Ollama 5. Chat interface 6. Citation support 7. Streaming response 8. Session memory 9. URL ingestion 10. PDF/DOCX/TXT support 11. Chunking and embeddings 12. Persistent vector storage 13. On-prem deployment 14. Fully local execution

The generated solution should resemble a professional enterprise internal knowledge assistant.

APPLICATION REQUIREMENTS

Build the following modules.

1. Frontend
- HTML
- Bootstrap 5
- Responsive design
- Chat window
- Upload panel
- URL ingest panel
- Citation display
- Streaming responses
- Modern UI styling
- JavaScript-based interaction
2. Backend Use FastAPI.

Required endpoints:

- /upload
- /ingest_url
- /chat
- /chat/reset
- /delete
- /
3. Document Processing Support:
- PDF
- DOCX
- TXT

Requirements:

- Extract text cleanly
- Preserve headings if possible
- Preserve page markers for PDFs
- Read table text from DOCX
- Clean duplicate whitespace
- Handle bad encoding gracefully
4. URL Ingestion
- Download HTML pages
- Extract readable text using BeautifulSoup
- Support PDF URLs
- Detect PDF links inside webpages

- Prevent SSRF attacks
- Block localhost and loopback addresses
- Validate URLs

5. Chunking Implement smart chunking:

- Configurable chunk size
- Configurable overlap
- Preserve semantic boundaries if possible
- Store metadata:
    - filename
    - page number
    - heading
    - chunk index
    - source URL

6. Embeddings Use Ollama embeddings.

Example embedding model:

- nomic-embed-text

Store embeddings locally.

7. Vector Database Use ChromaDB.

Requirements:

- Persistent local storage
- Metadata support
- Similarity search
- Top-K retrieval

8. LLM Generation Use Ollama chat API.

Suggested models:

- gpt-oss:latest

Requirements:

- Streaming response support
- Configurable temperature
- Configurable top-k retrieval
- Context injection
- Proper prompt engineering

9. RAG Pipeline Build complete retrieval pipeline:

- Query embedding

- Similarity search
- Context assembly
- Prompt construction
- LLM response generation
- Citation mapping

10. Session Memory

- Maintain conversation history
- Session IDs
- Reset session option
- Store chat memory in RAM

11. Citations Return citations like:

- filename
- page number
- chunk number
- heading

12. Security Implement:

- File type validation
- Safe filenames
- URL validation
- Upload size limits
- Error handling
- Graceful exception handling

13. Deployment Provide:

- requirements.txt
- setup instructions
- Ollama commands
- run commands
- Windows instructions
- troubleshooting guide

DESIRED PROJECT STRUCTURE

Generate the project in a clean modular structure similar to:

project_root/ | ├── app.py ├── rag.py ├── text_extract.py ├── url_ingest.py ├── requirements.txt ├── run.txt ├── storage/ ├── uploads/ ├── templates/ | └── index.html ├── static/ | ├── app.js | └── styles.css └── chroma_db/

Explain the role of every file.

IMPORTANT IMPLEMENTATION DETAILS

The application should:

- Use FastAPI backend.
- Use Bootstrap frontend.
- Use vanilla JavaScript.
- Use fetch() API.
- Use async endpoints where appropriate.
- Support response streaming.
- Use Ollama REST API.
- Use Chroma persistent client.
- Support incremental document indexing.
- Allow deleting indexed documents.
- Store document registry JSON.

Implement:

- Smart prompt templates
- Retrieval reranking if needed
- Context window management
- Token-safe prompt assembly
- Clean answer formatting
- Markdown rendering in frontend

PROMPT ENGINEERING REQUIREMENTS

Design a professional RAG prompt template.

The assistant should:

- Answer ONLY from retrieved context when possible.
- Clearly say when information is unavailable.
- Avoid hallucinations.
- Use conversational tone.
- Format answers cleanly.
- Use bullets and headings.
- Cite sources.
- Maintain continuity.

Include:

- System prompt
- Context formatting strategy
- Citation formatting strategy

- Conversation memory integration

UI REQUIREMENTS

Design a modern interface similar to ChatGPT:

- Left panel:
  - Upload files
  - URL ingest
  - Indexed docs list
- Right panel:
  - Chat interface
  - Streaming answers
  - Citations
  - Controls

Frontend features:

- Responsive layout
- Message bubbles
- Markdown rendering
- Loading spinners
- Smooth scrolling
- Session persistence using localStorage

DEVELOPMENT STYLE

VERY IMPORTANT:

Generate the solution incrementally.

For every phase: 1. Explain the concept. 2. Explain architecture. 3. Explain why the approach is chosen. 4. Then generate code. 5. Then explain the code. 6. Then explain how to test.

Do NOT dump everything at once.

Develop phase-by-phase.

Suggested phases:

Phase 1:

- Architecture planning
- Project setup
- Dependency installation

Phase 2:

- FastAPI basic app
- HTML frontend

Phase 3:

- File upload system

Phase 4:

- Text extraction

Phase 5:

- Chunking and embeddings

Phase 6:

- ChromaDB integration

Phase 7:

- Ollama integration

Phase 8:

- RAG pipeline

Phase 9:

- Streaming responses

Phase 10:

- Citations

Phase 11:

- Session memory

Phase 12:

- URL ingestion

Phase 13:

- UI polishing

Phase 14:

- Deployment and optimization

DEBUGGING REQUIREMENTS

Whenever generating code:

- Add logging.
- Add comments.
- Add exception handling.
- Explain common errors.
- Explain fixes.

If dependencies are missing:

- Explain installation.
- Explain alternatives.

OPTIMIZATION REQUIREMENTS

The solution must be optimized for modest hardware.

Include:

- Efficient chunk sizes
- Lazy loading where possible
- Efficient retrieval
- Memory-efficient processing
- CPU-friendly defaults

ADVANCED FEATURES (OPTIONAL)

After the basic system works, optionally help implement:

- Multi-user sessions
- Authentication
- Role-based access
- WebSocket streaming
- OCR support
- Image extraction from PDFs
- Table-aware retrieval
- Semantic caching
- Background indexing

HOW TO RESPOND

You must:

- Act like a mentor + senior engineer.
- Be highly detailed.
- Never skip implementation details.

- Never assume hidden knowledge.
- Generate complete working code.
- Ensure all imports are included.
- Ensure filenames are clearly mentioned.
- Ensure folder structure is maintained.
- Ensure code compatibility.
- Ensure beginner-friendly explanations.

Start with: 1. Overall architecture explanation 2. Data flow diagram explanation 3. Technology stack reasoning 4. Complete folder structure 5. Dependency list 6. Then begin Phase 1